

420-126-LI

POO 1

Scanner, portée des variables et constantes

Plan

- Lire une donnée entrée par l'utilisateur (scanner)
- La portée des variables
- Constantes

Lire une donnée entrée par L'utilisateur (Scanner)

- Pour lire une donnée fournie par l'usager, le code devra obligatoirement utiliser une **variable** pour être en mesure d'utiliser la valeur lue, par la suite.

1 usage

```
public void lireEtAfficherNomPrenom() {  
    Scanner sc; //Instance de l'utilitaire de lecture  
    String nom;  
    String prenom;  
  
    sc = new Scanner(System.in); //Instanciation de l'utilitaire de lecture  
  
    System.out.println("Entrez votre nom : ");  
  
    //Appel d'une méthode qui récupère une chaîne de caractères saisis au clavier.  
    nom = sc.nextLine();  
  
    System.out.println("Entrez votre prénom : ");  
    prenom = sc.nextLine(); //lecture du prénom  
  
    System.out.println("Bonjour Monsieur ou Madame " + nom);  
    System.out.print("Je sais que votre prénom est " + prenom);  
}
```

```
"C:\Program Files\Java\jdk-18.0.1.1\bi  
Entrez votre nom :  
Brunet  
Entrez votre prénom :  
Julien  
Bonjour Monsieur ou Madame Brunet  
Je sais que votre prénom est Julien  
Process finished with exit code 0
```

Lire une donnée entrée par L'utilisateur (Scanner)

- Selon le même modèle on peut utiliser `nextInt()` pour lire des entiers, etc...

```
public void lireEtAfficherDesNombresDeBilles() {  
    Scanner sc; //Instance de l'utilitaire de lecture  
    int nbBillesDenis;  
    int nbBillesLouise;  
    int totalBilles;  
  
    sc = new Scanner(System.in); //Instanciacion de l'utilitaire de lecture  
  
    System.out.println("Entrez le nombre de billes de Denis : ");  
    nbBillesDenis = sc.nextInt();  
  
    System.out.println("Entrez le nombre de billes de Louise : ");  
    nbBillesLouise = sc.nextInt();  
  
    totalBilles = nbBillesDenis + nbBillesLouise;  
  
    System.out.println("Denis et Louise ont " + totalBilles + " billes ensemble.");  
}
```

- **Attention** au `nextLine()` après un `nextInt()` qui se comporte de manière surprenante... Il faut être rusé pour contourner le problème.

Portée des variables

- Selon l'endroit où l'on déclare une variable, celle-ci pourra être accessible (visible) de partout dans le code ou bien uniquement dans une portion de celui-ci, on parle alors de **portée d'une variable**.
- En java, toute variable est déclarée à l'intérieur d'un groupe d'accolades { }, un bloc de code. **Sa portée se restreint donc à l'intérieur de ce bloc**. Le groupe d'accolades peut être une méthode ou d'autres structures que nous verrons bientôt.
- **Important** : Une variable est disponible pour son utilisation uniquement après avoir été déclarée.
- **Important** : Il est interdit d'avoir deux variables de même nom si elles ont une portée commune.

Portée des variables (exemple)

```
public class Exemple4_PorteeVariables {  
  
    public static void main(String[] args) {  
        //a n'est valide que dans le main  
        int a = 3;  
        //int a = 13;  
        Exemple4_PorteeVariables objet = new Exemple4_PorteeVariables();  
        System.out.println("Voici a : " + a );  
  
        objet.methode1();  
        objet.methode2();  
  
        //b et c sont inconnus ici!  
  
        if (a > 3){  
            int d = 10;  
            a += d;  
            System.out.println("Voici d : " + d );  
            System.out.println("Voici a : " + a );  
  
            //b, c et e sont inconnus ici!  
        }else{  
            int e = 10;  
            a -= e;  
            System.out.println("Voici e : " + e );  
            System.out.println("Voici a : " + a );  
  
            //b, c et d sont inconnus ici!  
        }  
    }  
}
```

SUITE du code de la classe programme (voir l'exemple de code fourni)

```
    public void methode2() {  
        //b n'est valide que dans cette methode  
        int b = 5;  
        b += 8;  
        System.out.println("Voici b : " + b );  
        //a et c sont inconnus ici!  
    }  
  
    public void methode1() {  
        //c n'est valide que dans cette methode  
        int c = 7;  
        System.out.println("Voici c : " + c );  
  
        // a et b sont inconnus ici!  
    }  
}
```

Constantes (1)

- En programmation Java, une constante est une valeur immuable, **qui ne peut pas être modifiée une fois qu'elle a été définie**.
- Les constantes sont largement utilisées pour stocker des valeurs fixes comme des nombres, des chaînes de caractères ou d'autres données qui **restent constantes tout au long de l'exécution** du programme.
- Pour déclarer une constante en Java, on utilise le mot-clé « **final** ».
Par exemple :

```
final double VITESSE_SON = 343 ;
```
- L'utilisation de la constante est ensuite facilitée par le nom significatif que vous lui avez donné (**EN_MAJUSCULE**, par convention).

Constantes (2)

- Parmi les avantages des constantes (**bonne pratique**), elles **améliorent la lisibilité du code** et permettent de **centraliser la gestion des valeurs fixes** (et donc de faciliter la maintenance du code).
- Une fois qu'une constante est définie, il n'est **pas possible de la modifier accidentellement**, ce qui contribue à éviter des erreurs potentielles dans le code.
- Notons qu'il est plus intéressant de définir une constante au niveau de la classe (au même niveau qu'on trouve les déclarations de méthode). À ce moment il faut ajouter « **public static** » et la constante est disponible partout dans le programme. **C'est une pratique courante à privilégier.**

```
public static final double VITESSE_LUMIERE = 29979458 ;
```


Constantes (Exemple)

```
public class TestUtilisationConstantes {
```

1 usage

```
public static final double VITESSE_LUMIERE = 29979458 ;
```

Julien.Brunet *

```
public static void main(String[] args) {
```

```
    final double VITESSE_SON = 343 ;
```

```
    System.out.println("Voici une constante définie dans de la méthode : " + VITESSE_SON);
```

```
}
```

Julien.Brunet *

```
public void test(){
```

```
    System.out.println("Voici une constante définie hors de la méthode (depuis n'importe ou! : " + VITESSE_LUMIERE);
```

```
    //On ne peut pas accéder à VITESSE_SON ici
```

```
}
```

```
}
```